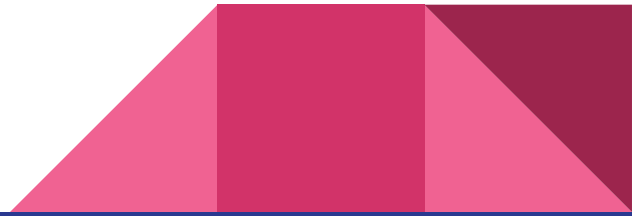
Unit 2.5 - From conceptual to logical

Transition to the Relational model

Introduction

After designing a database conceptually with an Entity-Relationship diagram (ERD), the next step is to implement it logically using the **Relational Database Model**.

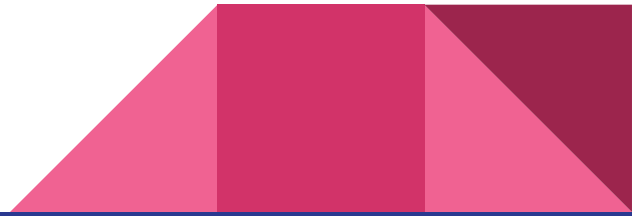
The relational model was originally proposed by Edgar F. Codd in **1970** and has since become the most widely used data model for databases.



Introduction

The relational model is a logical representation of the database, in contrast to the ER model which is a conceptual design tool.

In other words, an ER diagram serves as a blueprint of entities and relationships, while **the relational model translates that blueprint into actual tables and fields that can be implemented in a database system**.



Difference between E/R Diagram and Relational model

- **ERD** (Entity-Relationship Diagram): A **conceptual design**, visual model that **represents real-world** entities, their attributes, and their relationships at a high level, independent of any database software.

- **Relational Model**: A **logical**, **structural design** that organizes data into actual tables (relations) with rows, columns, primary keys, and foreign keys **to implement** the structure in a database management system (DBMS).

Data structuring

The following points must be specified:

- Every relationship must have a name that uniquely identifies it within the database.
- Each row stores the data we are interested in reflecting from a specific real-world object.
- Each column contains, in each cell, data of the same type, and it can be called attribute.
- Each cell, or intersection between row and column, can store a single value.

Data structuring

The table reflects the tabular structure of the relationship STUDENT containing personal data relating to individuals enrolled in a teaching center.

Each row contains a number of related data which, in this case, are those that belong to the same student.

The relationship has a name (STUDENT), like each of the columns (DNI, Name, Surname and Telephone). If these names are meaningful enough, they allow you to immediately grasp the meaning of the values of the data stored in the relationship.

Table: 1.1. Example of relationship

STUDENTS			
identity card	**First name**	**Surnames**	**Phone**
47126654F	Josep	Bel Rovira	453641282
51354897S	Anna	Pacheco Cuscó	723352151
56354981L	Xavier	Rius Montalvo	726922235

Schema

The schema of a relation is a name that uniquely identifies within the database, and the set of attributes contained inside.

The schema of the relation shown in the table would look like this:

STUDENT (DNI, Name, Surname, Telephone)

It should be noted that the order in which the attributes are shown to us is indifferent.

Table: 1.2. Example of relationship

STUDENTS			
identity card	**First name**	**Surnames**	**Phone**
47126654F	Josep	Bel Rovira	453641282
51354897S	Anna	Pacheco Cuscó	723352151
56354981L	Xavier	Rius Montalvo	726922235

Control

The following are the characteristics of one of the most **common notation systems**:

It is necessary to write the **name of the relations with capital letters** and preferably in singular.

The name of the **attributes** must be written **starting with a capital letter and continuing with a lowercase** letter, as long as it is not an acronym, as it is then more convenient to leave all the letters in capital letters (such as DNI). In order to make compound names more readable, you can headline each of the words that make up the field name with a capital letter (for example: BirthDate, PrivatePhone, etc.). - This is known as **PascalCase**

The attributes of a relationship are unique within it. Their name cannot be repeated within the same relationship. However, different relationships can contain attributes with the same name. On the other hand, it must be said that the domains of different attributes of the same relationship can be identical, although the respective fields store the values of different properties of the object.

Control

Sometimes the **attributes** of the relationships may not contain any values or, in other words, they **may contain null values**.

The degree of a relationship depends on the number of attributes.
The relationship with the STUDENT scheme (DNI, Name, Surname, Telephone) is grade 4, because it has four attributes.

The cardinality of a relation is given by the number of rows that form the extension.
the cardinality of the STUDENT relationship is 3, because its extension contains three rows corresponding to the three students who are currently enrolled.

Extension

The extension of a relation consists of data values stored in all the rows.

If we take as a basis, once again, the relationship with the STUDENT scheme (DNI, Name, Surname, Telephone) in table 2, its extension would be a list in which all the students in the database would appear:

Student 1: 47126654F, Josep, Bel Rovira, 453641282 Student 2: 51354897S, Anna, Pacheco Cuscó, 723352151 Student 3: 56354981L, Xavier, Rius Montalvo, 726922235

Candidate keys, primary key and alternative keys

Candidate keys are the attributes or sets of attributes that allow to identify the rows contained in the extension. However, when implementing a DB, only one of the candidate keys in each relationship must be chosen. That is the primary key.

We will easily see how the attributes DNI, NumSS (Social Security number) and NumMatricula, being personal and unrepeatable, could help us to uniquely identify students. Therefore, they would be key candidates. If, for example, you choose DNI as the primary key:
STUDENTS (DNI , NumSS, Matriculation Number, Name, Surname, Telephone)

Table: 1.4. Example of relation with null values

STUDENTS			
identity card	**First name**	**Surnames**	**Phone**
47126654F	Josep	Bel Rovira	453641282
51354897S	Anna	Pacheco Cuscó	723352151
56354981L	Xavier	Rius Montalvo	726922235
24583215W	Mariona	Castellví Mur	NULL

Candidate keys, primary key and alternative keys

Any relational DB, no matter how small, usually contains more than one relationship. In order to properly reflect the links between some real-world objects, the rows of the different relationships in a database need to be able to interrelate.

A **foreign key** consists of an attribute, or set of attributes, of the schema of a relationship, which is **used to relate its rows to the rows of another** relationship in the database.



Candidate keys, primary key and alternative keys

The STUDENT relationship, as shown in Table 1.5 , incorporates two **foreign keys**.

One of them, CodiAula, refers to the **primary key of the CLASSROOM** relationship (formed by the Codi attribute), set out in Table 1.6 , in order to indicate which classroom corresponds to each student.

Instead, DNIDelegate refers to the **primary key of the same relationship** (formed by the DNI attribute), and serves to indicate which delegate each student is represented by.

Note that the student Mariona Castellví has not yet been assigned a delegate or classroom and, for this reason, the row that represents her currently contains **null** values in the attributes of the two foreign keys.

Table: 1.5. Example of relationship with foreign keys

STUDENTS					
identity card	**First name**	**Surnames**	**Phone**	**DNIDelegate**	**CodiAula**
47126654F	Josep	Bel Rovira	453641282	47126654F	102
51354897S	Anna	Pacheco Cuscó	723352151	51354897S	201
56354981L	Xavier	Rius Montalvo	726922235	51354897S	201
24583215W	Mariona	Castellví Mur	NULL	NULL	NULL

Table: 1.6. Example of relationship with referenced primary key

CLASSROOM	
Codi	**Capacity**
101	40
102	36
201	30

Candidate keys, primary key and alternative keys

The most common notation for designating **foreign keys** to relationships is

STUDENT (DNI , FirstName, Surnames, Phone, DNIDelegate, CodiAula) WHERE {DNIDelegate} REFERENCE STUDENT and {CodiAula} REFERENCE CLASSROOM

CLASSROOM (Codi , Capacity)



ER to relational

The ER model is based on the entities and the interrelationships between them. We can advance a few general aspects on how these elements should be translated into the relational model:

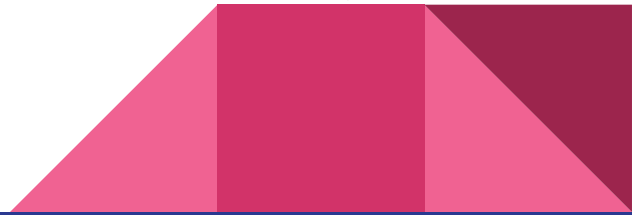
- **Entities always give rise to relationships**
- **1-1** connectivity binary interrelationships **originate foreign keys in existing relationships**
- **M-N** connectivity binary interrelationships always transformed into **new relationships**
- **1-N connectivity binary interrelationships originate foreign keys or be transformed into new relation, depending on the case**



ER to relational: Entities to relations (tables)

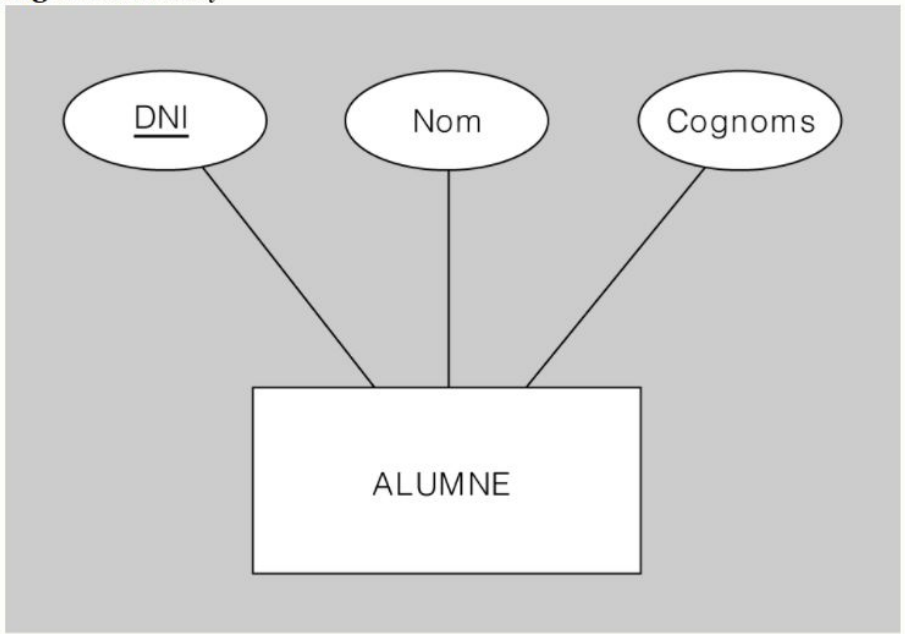
Each entity of an ERD is transformed into a relation (table) in the relational model:

- The attributes of the original entity will be the attributes of the resulting relation (table).
- The primary key of the original entity will be the primary key of the resulting relation (table).
- When an entity intervenes in some 1-1 or 1-N binary interrelationship, it may be necessary to add new attributes later, in order for them to act as keys outside the relationship (identify interrelationships with Foreign Keys).



ER to relational: Entities

Figure 1.1. Entity

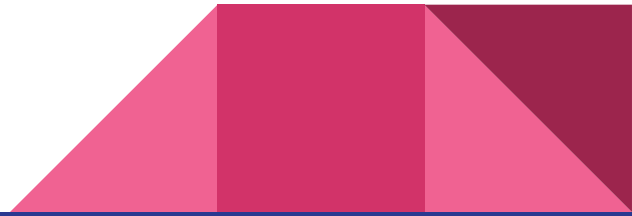


STUDENT (DNI , Name, Surname)

ER to relational: Relationships

Once all the entities have been transformed into relationships, it is necessary to **translate the interrelationships** in which they participate.

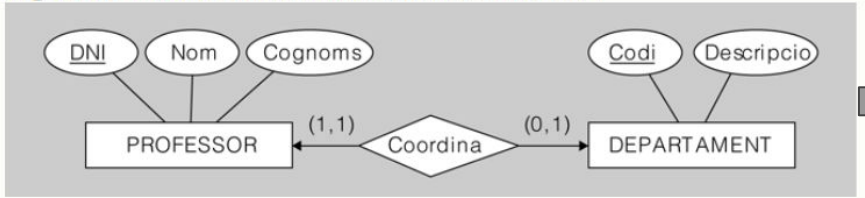
To translate binary interrelationships one must take into account their connectivity as well as existing dependencies (participation).



ER to relational: Relationships (1:1)

A foreign key that refers to the other relationship must be added to either relationship. But if in the relationship one entity is optional and the other mandatory, then the optional side must host the foreign key with a NOT NULL constraint.

Figura 1.2. Interrelació binària amb connectivitat 1-1



Both optional or both mandatory:

DEPARTMENT (Code , Description)

PROFESSOR (DNI , Name, Surname, DepartmentCode)
WHERE {Department Code} REFERENCES DEPARTMENT

One optional, One mandatory (like the diagram):

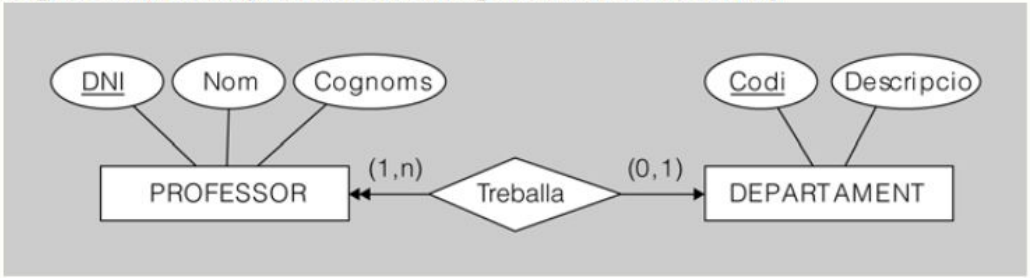
PROFESSOR (DNI , Name, Surname)

DEPARTMENT (Code , Description, DNIProfessor (NOT NULL)) WHERE {DNIProfessor} REFERENCES PROFESSOR

ER to relational: Relationships (1:N)

Standard procedure: a foreign key must be added to the table corresponding to the entity located on the N side of the interrelationship.

Figure 1.3. Binary interrelationship with 1-N connectivity



DEPARTMENT (Code , Description)

PROFESSOR (DNI , Name, Surname, DepartmentCode)
WHERE {DepartmentCode}
REFERENCES DEPARTMENT

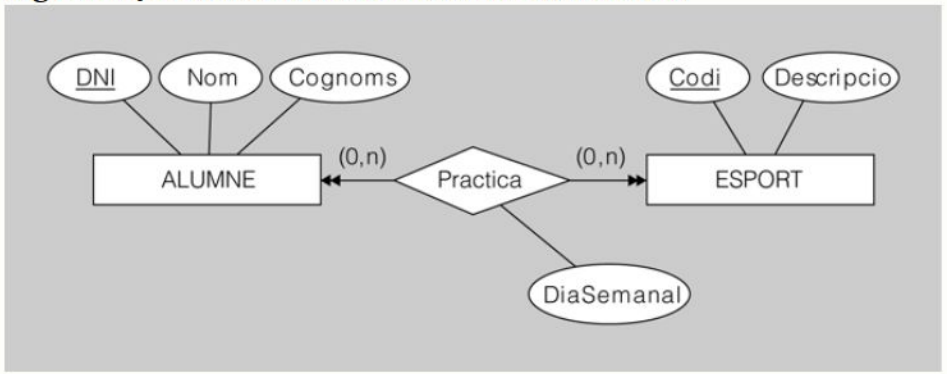
Note: DepartmentCode as a Foreign Key might contain NULL values

ER to relational: Relationships (M:N)

Each M:N interrelationship is **transformed into a new relationship** with the following characteristics:

- Its primary key will be formed by the primary keys of the two interrelated entities.
- The attributes of the interrelationship (if any) will become attributes of the new table.

Figura 1.4. Interrelació binària amb connectivitat M-N



STUDENT (DNI, Name, Surname)

SPORT (Code , Description)

PRACTICE (DNIAlumne, CodeSport, DiaSemanal) WHERE {DNIAlumne} REFERENCES STUDENT and {CodeSport} REFERENCES SPORT

ER to relational: Ternary relationships

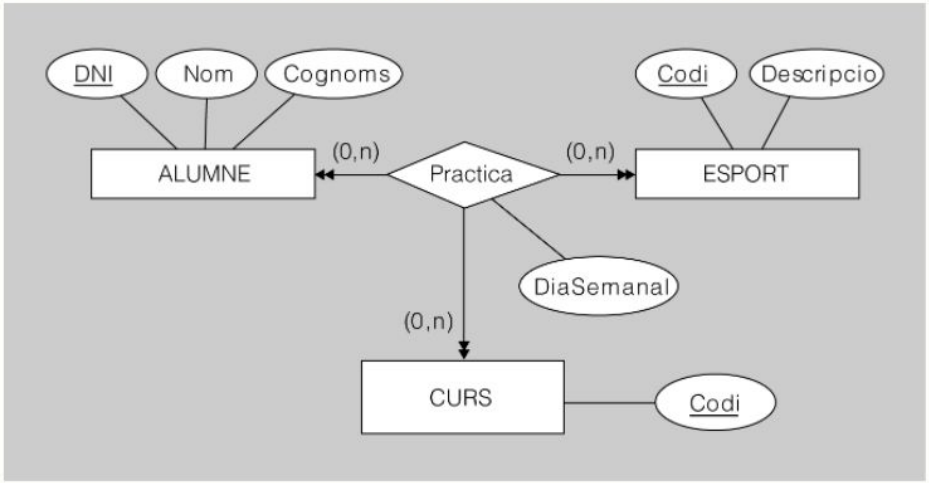
The basic rule for ternary relationships is:

- **Transform the relationship into a new table.**
- **Primary key**: Depends on each individual case. The primary key is *usually* formed by combining the primary keys of the three participating entities. However, the cardinality constraints may allow a smaller combination of these keys to be used as the primary key.
- **Foreign keys:** The primary keys of the three participating entities are included in the new table as foreign keys.
- **Relationship attributes:** Any attributes belonging to the ternary relationship are also included in the new table.

ER to relational: Ternary relationships (M:N:P)

In this case, the primary key is made up of all the attributes that make up the primary keys of the three interrelated entities.

Figure 1.5. Ternary interrelationship with MNP connectivity



STUDENT (DNI , Name, Surname)

SPORT (Code , Description)

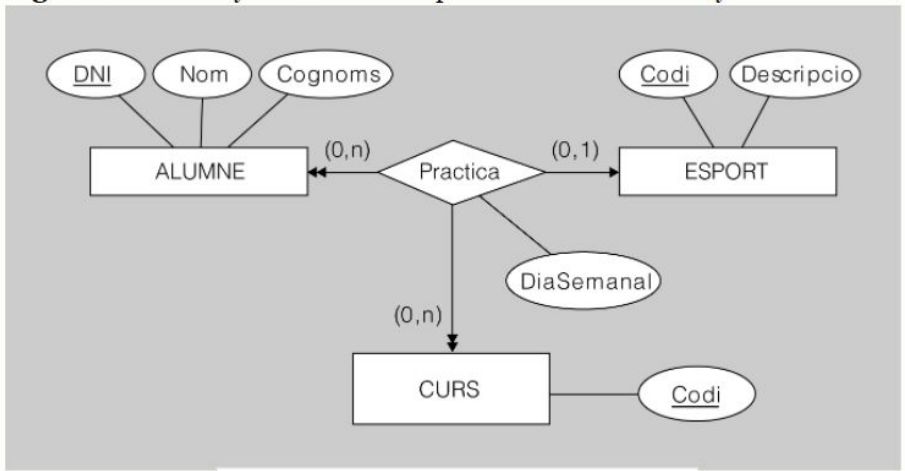
COURSE (Code)

PRACTICE (DNIAlumne, CodiEsport, CodiCurs, DiaSemanal) WHERE {DNIAlumne} REFERENCES STUDENT {CodiEsport} REFERENCES SPORT and {CodiCurs} REFERENCES CURS

ER to relational: Ternary relationships (1:M:N)

The primary key is composed of all the attributes that make up the primary keys of the two entities that are on both sides of the interrelation labeled with an N

Figure 1.6. Ternary interrelationship with 1-MN connectivity



STUDENT (DNI , Name, Surname)

SPORT (Code , Description)

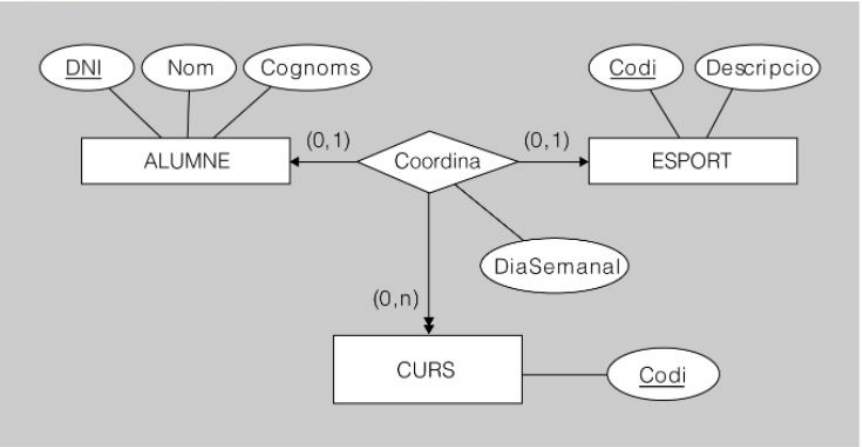
COURSE (Code)

PRACTICE (DNIAlumne , CodiSport, CodiCurs , DiaSemanal) WHERE {DNIAlumne} REFERENCES STUDENT and {CodiSport} REFERENCES SPORT and {CodiCurs} REFERENCES CURS

ER to relational: Ternary relationships (1:1:N)

In these cases, the primary key is composed of the attributes that form the primary key of the entity on the N side of the interrelationship, plus the attributes that form the primary key of either of the other two entities connected with cardinality 1.

Figure 1.7. Ternary interrelationship with 1-1-N connectivity



STUDENT (DNI , Name, Surname)

SPORT (Code , Description)

CURS (Code)

Coordination (CodiCurs, DNIAlumne , CodiEsport, DiaSemanal)
WHERE {CodiCurs} REFERENCES CURS and {DNIAlumne} REFERENCES STUDENT and {CodiEsport} REFERENCES SPORT

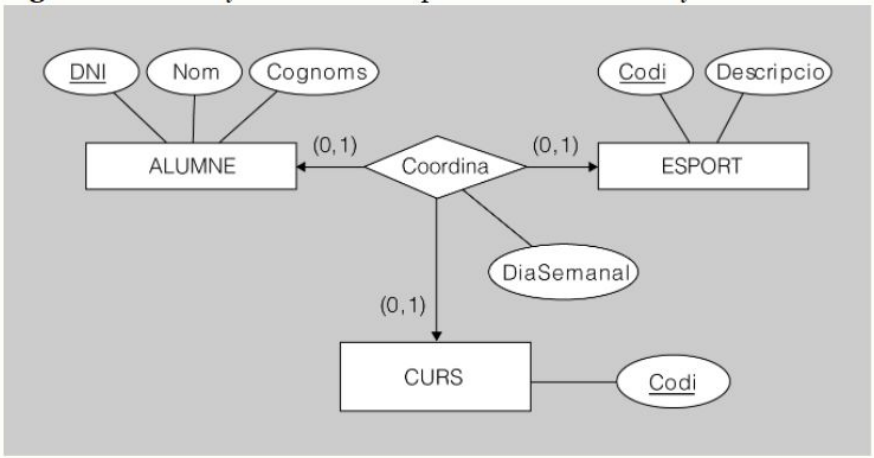
Or:

Coordination (CodiCurs, DNIAlumne , CodiEsport, DiaSemanal)
WHERE {CodiCurs} REFERENCES CURS and {DNIAlumne} REFERENCES STUDENT and {CodiEsport} REFERENCES SPORT

ER to relational: Ternary relationships (1:1:1)

In these cases, the primary key is composed of the attributes that form the primary key of any two entities, since all three are connected with cardinality 1.

Figure 1.8. Ternary interrelationship with 1-1-1 connectivity



STUDENT (DNI , Name, Surname)

SPORT (Code , Description)

CURS (Code)

Coordination (CodiCurs, DNIAlumne , CodiEsport, DiaSemanal) WHERE {CodiCurs} REFERENCES CURS and {DNIAlumne} REFERENCES STUDENT and {CodiEsport} REFERENCES SPORT

Or:

Coordination (CodiCurs, DNIAlumne , CodiEsport, DiaSemanal) WHERE {CodiCurs} REFERENCES CURS and {DNIAlumne} REFERENCES STUDENT and {CodiEsport} REFERENCES SPORT

Or:

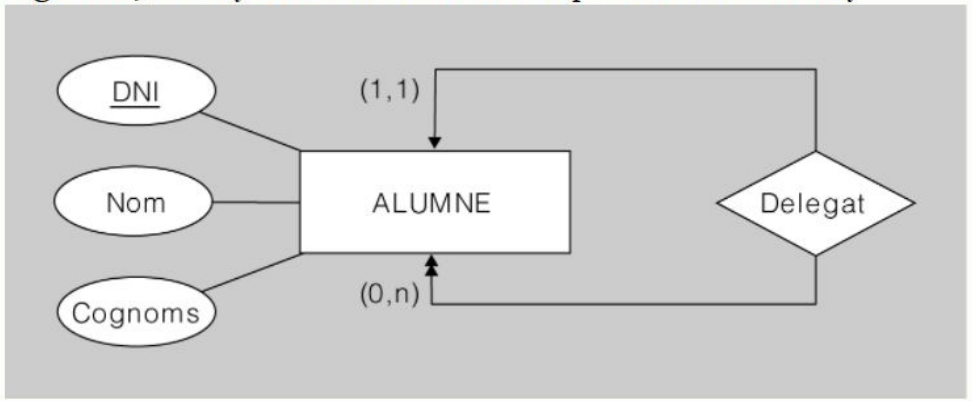
Coordination (CodiCurs, DNIAlumne , CodiEsport, DiaSemanal) WHERE {CodiCurs} REFERENCES CURS and {DNIAlumne} REFERENCES STUDENT and {CodiEsport} REFERENCES SPORT

ER to relational: Recursive relationships (1:1) or (1:N)

In these situations, it is necessary to add to the relationship arising from the original entity that relates to itself as a foreign key that refers to the primary key itself.

Obviously, the attributes of the **foreign key cannot have the same names** as those of the primary key to which they refer, since they are both in the same relationship, and this would violate the principles of the relational model.

Figure 1.9. Binary recursive interrelationship with 1-N connectivity



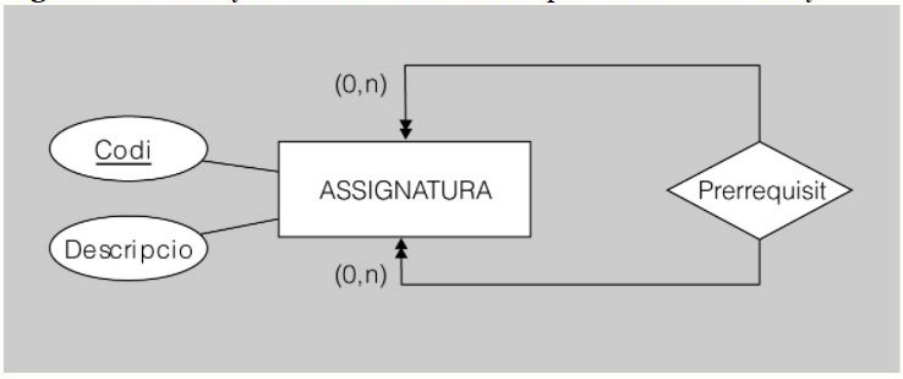
STUDENT (DNI , Name, Surname, DNIDelegate) WHERE {DNIDelegate} REFERENCE STUDENT

ER to relational: Recursive relationships (M:N)

When the binary recursive interrelation has M:N connectivity a new relationship (Table) is originated, which has as primary key the attributes that form the primary key of the original entity, but **twice**, since it is necessary to model the fact that the only entity which intervenes in the intended conceptualization interrelates with itself (and not with a different one).

It is necessary to conveniently modify the names of these attributes that are present twice in the new relationship so that they do not match, and thus respect the guidelines of the relational model.

Figure 1.10. Binary recursive interrelationship with MN connectivity



ASSIGNATURA (Code, Description)

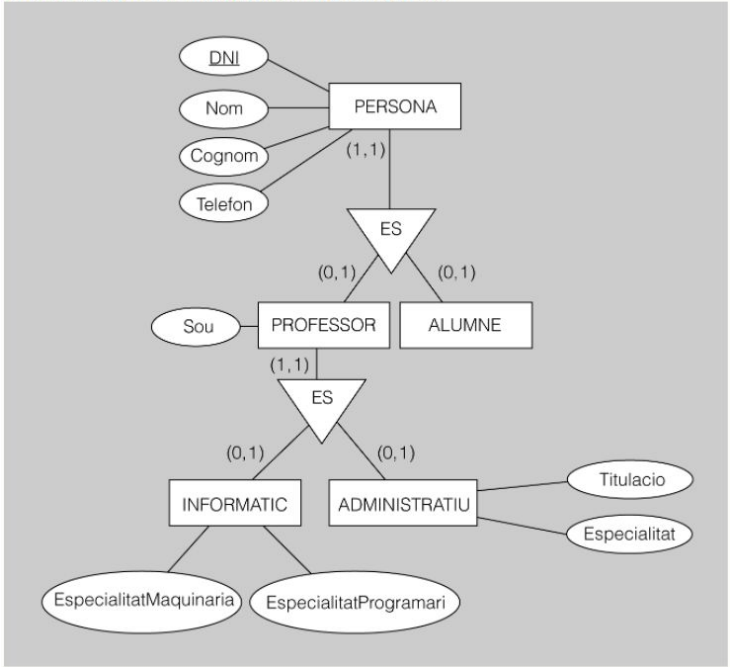
PREREQUISITE (IDAssignatura, IDAssignaturaRequerida) WHERE {IDAssignatura} REFERENCES ASSIGNATURA and {IDAssignaturaRequerida} REFERENCES ASSIGNATURA

ER to relational: Inheritance

In these cases, both the superclass entity and the subclass type entities are transformed into new relationships.

The relationship derived from the superclass inherits from it the primary key. In addition, it is responsible for storing attributes common to all specialization or generalization.

Figure 1.13. Generalization and specialization



PERSON (DNI , Name, Surname, Telephone)

PROFESSOR (DNI , Salary) WHERE {DNI} REFERENCE PERSON

STUDENT (DNI) WHERE {DNI} REFERENCES PERSON

COMPUTERSCIENTIST (DNI, SoftwareSpecialty, HardwareSpecialty) WHERE {DNI} REFERENCES PROFESSOR

ADMINISTRATIVE (DNI , Degree, Specialty) WHERE {DNI} REFERENCES PROFESSOR